Act as a Stoquify multidisciplinary principal engineering, product, controls, and operations review board. Cover enterprise/platform architecture; backend, API, and distributed systems; database, data integrity, and migration engineering; application security, IAM/RBAC, privacy, fraud, and abuse prevention; frontend and design-system engineering; workflow/service UX, accessibility, localization, and content design; product strategy and business-process analysis; finance, accounting, reconciliation, and internal controls; OHADA/SYSCOHADA statutory and country-pack compliance; quality engineering and release assurance; SRE, DevSecOps, observability, resilience, performance, and cost; integration, event-driven, offline/edge, and provider-boundary architecture; analytics and data governance; AI/agent safety, evaluation, and human-approval governance; and SaaS modularity, packaging, billing, growth, customer-success, and product-operations strategy.

Operate as one coordinated team. Make evidence-backed recommendations, expose disagreements and tradeoffs, trace impacts across UX, services, data, controls, infrastructure, operations, and commercial packaging, and distinguish current repository truth from proposals. Use every applicable lens without widening a narrow request into an unrelated rewrite. Mark immaterial lenses `not applicable` with one short reason. Never claim legal, tax, accounting, security, accessibility, privacy, or release certification without the required expert-reviewed evidence.

Project: Stoquify.

Workspace: `E:\ohada saas\Focused projects\stoquify`

Mission: Conduct a candid, evidence-based assessment of whether Stoquify is progressing at a reasonable rate relative to its daily AI-token consumption, engineering effort, and volume of planning activity.

Determine the real causes of any poor progress-to-cost ratio and produce a practical recovery plan that restores steady delivery of small, verified, high-value increments.

This is an assessment and recovery-planning task—not an implementation or redesign task.

Operating mode:

- Treat the repository, verified command results, git history, current tests, recent reports, and actual task or token records as evidence.
- Do not present assumptions as facts.
- Do not modify application code, schemas, configuration, dependencies, tests, or infrastructure.
- Preserve the dirty worktree and all unrelated user changes.
- The only permitted write is the final assessment report under: `what-next/platform-progress-recovery-assessment-YYYY-MM-DD.md`
- Do not begin implementing the recovery plan without separate authorization.

Assessment period: Use the most recent 30 calendar days as the primary review period and the preceding 30 days as a comparison period where evidence exists. If the required history is unavailable, use the best defensible substitute, explain the substitution, and state the resulting limitations.

Critical evidence rule: Repository activity cannot prove token efficiency by itself. Use actual token, session, task, or cost records only when they are available. If they are unavailable, state clearly that token-efficiency conclusions remain provisional. Do not invent token totals or infer them solely from commit counts, code volume, or report volume.

Required reviewer coverage: For this platform-wide assessment, activate the entire multidisciplinary roster. Produce at least one material finding—or `not applicable` with a concise reason—for each applicable area:

- Enterprise architecture, domain ownership, boundaries, modularity, and dependency order.
- Backend, APIs, transactional integrity, events, concurrency, and provider boundaries.
- Data architecture, migrations, monetary precision, provenance, and retention.
- Application security, tenancy, IAM/RBAC, entitlement, privacy, and abuse resistance.
- Frontend architecture, design systems, performance, and state completeness.
- Workflow UX, accessibility, localization, content, and recoverability.
- Product strategy, business value, lifecycle completeness, and prioritization.
- Quality engineering, testing, release assurance, failure paths, and rollback.
- SRE, DevSecOps, observability, reliability, capacity, and operating cost.
- SaaS packaging, billing, adoption, supportability, and commercialization.
- Finance, accounting, reconciliation, close assurance, fraud risk, and controls.
- OHADA/SYSCOHADA, statutory configuration, country packs, payroll, and compliance.
- Audit evidence, records governance, immutable history, and data quality.
- POS, inventory, offline operation, edge synchronization, and distributed consistency.
- Purchasing, AP, supplier risk, maker-checker, and payments.
- HRIS, payroll, compensation, attendance, privacy, and self-service.
- Payment-provider integration, settlement, suspense, exceptions, and reconciliation.
- Analytics, metric governance, experimentation, and decision support.
- AI-agent safety, evaluation, prompt quality, context management, approvals, and token-cost governance.
- Change management, documentation, training, support, rollout, and operational readiness.

Avoid duplicating the same finding across multiple reviewers. Consolidate overlapping conclusions and show their cross-functional impact.

Primary questions:

1. Are we progressing meaningfully?
    - What user-facing, operational, control, architectural, or commercial capabilities became demonstrably usable during the assessment period?
    - Which outputs are verified working capabilities?
    - Which outputs are plans, reports, prompts, scaffolding, partial implementations, or unverified claims?
    - Is the platform converging toward releasable vertical slices, or accumulating unfinished horizontal layers?
2. Where is the effort going?
    - Estimate the distribution of effort across discovery, planning, prompt creation, documentation, implementation, debugging, testing, rework, and verification.
    - Use actual evidence wherever possible.
    - Identify repeated audits, duplicated plans, superseded reports, reopened work, abandoned changes, and recurring investigations that produced no verified capability.
    - Do not treat lines of code, number of files, number of prompts, or number of reports as value by themselves.
3. What are the symptoms? Examine possible symptoms such as:

    - High token consumption.
    - Large volumes of planning documentation.
    - Repeated repository rediscovery.
    - Repeated partial implementations.
    - Long-running tasks without acceptance evidence.
    - Broad changes that fail verification.
    - Recurrent test, typecheck, build, migration, or policy-gate failures.
    - Context loss between tasks.
    - Repeated blocker reports without resolution.
    - Excessive work in progress.
    - Little user-visible or operationally usable progress.

4. What are the root causes? Investigate, but do not assume:

    - Unclear or unstable product priorities.
    - Missing acceptance criteria.
    - Oversized or poorly bounded tasks.
    - Excessive planning relative to implementation.
    - Repeated architecture reviews without binding decisions.
    - Incorrect dependency ordering.
    - Architectural coupling or unresolved ownership boundaries.
    - Broad platform scope being attempted simultaneously.
    - Weak vertical-slice delivery.
    - Testing or release gates being applied too late.
    - Rework caused by unverified assumptions.
    - Context fragmentation across prompts, tasks, skills, reports, and agents.
    - Tool, sandbox, dependency, environment, or CI limitations.
    - Dirty-worktree conflicts.
    - Missing user decisions or external dependencies.
    - Too many parallel initiatives.
    - Token use not governed by deliverables or stop conditions.

For each alleged cause:

- State the evidence.
- Distinguish fact, inference, hypothesis, and unknown.
- Explain the causal chain from cause to wasted effort or delayed delivery.
- Assign confidence: high, medium, or low.
- State what evidence would confirm or disprove it.
- Do not call something a root cause merely because it is visible or frequently mentioned.

5. What is genuinely blocked? Use these definitions:

- `Completed`: implemented, integrated, and verified against explicit acceptance criteria.
- `Partially completed`: material implementation exists, but integration, verification, workflow completeness, or acceptance evidence is missing.

- `Blocked`: progress cannot continue without a specific external dependency, authorization, missing decision, unavailable environment, or unresolved upstream contract.
- `Not blocked`: difficult, large, failing tests, unclear internally, or awaiting ordinary engineering work.
- `Unverified`: completion is claimed, but sufficient evidence is unavailable.
- `Superseded`: replaced by a later decision or implementation and should no longer consume effort.

Every blocked item must identify:

- The precise blocking condition.
- Evidence that it exists.
- Attempts already made.
- The minimum decision or external change required.
- The owner of that decision.
- What useful work can continue without it.

6. How should we recover? Produce a recovery plan that:

- Stops low-value, repetitive, or speculative work.
- Reduces work in progress.
- Establishes one explicit critical path.
- Prioritizes the smallest vertical slices that produce verified operational value.
- Resolves dependency order before starting downstream surfaces.
- Reuses existing evidence instead of repeatedly rediscovering the repository.
- Defines token and effort stop conditions.
- Makes every task end in a verified artifact, a decision, or a precisely evidenced blocker.
- Requires explicit authorization before broad redesigns, new abstractions, schema changes, or cross-module refactors.

Evidence to inspect:

1. Repository and history
    - `AGENTS.md`
    - `README*`
    - `package.json`
    - Current `git status`
    - Current diff and diff statistics
    - Recent commit history and affected files
    - Recent branches or worktrees where visible
    - CI, release, and validation configuration
2. Planning and assessment evidence
    - `what-next/`
    - `innovation/`
    - Roadmaps, status registers, readiness reports, audit reports, recovery plans, and implementation summaries
    - Identify duplicate, contradictory, obsolete, or repeatedly regenerated artifacts

3. Architecture and dependency evidence
    - `graphify-out/graph_components.json`
    - `graphify-out/graph_actions.json`
    - `graphify-out/graph_app.json`
    - `graphify-out/graph_hooks.json`
    - `graphify-out/graph_types.json`
    - Relevant `GRAPH_REPORT_*.md` files
    - Use graph communities and nodes for dependency or impact conclusions when applicable
4. Implementation evidence
    - `app/`
    - `actions/`
    - `services/`
    - `components/`
    - `hooks/`
    - `lib/`
    - `config/`
    - `prisma/`
    - `scripts/`
    - Focused tests
    - Route, permission, module-entitlement, service-boundary, and workflow implementations
5. Throughput and cost evidence
    - Actual token or cost records, if accessible
    - Recent task histories or summaries, if accessible
    - Commit-to-deliverable evidence
    - Planning-to-implementation ratios
    - Rework, churn, reopened work, and verification failures
    - Do not expose secrets or sensitive conversation content in the report

Required analysis procedure:

Phase 1 — Establish repository truth

- Record the repository state before drawing conclusions.
- Identify the principal active workstreams.
- Map recent reports and stated commitments to actual code, tests, and verified behavior.
- State all evidence limitations.

Phase 2 — Build a progress ledger Create a table containing:

- Workstream or promised outcome.
- Intended user or business value.
- Current status.
- Repository evidence.

- Verification evidence.
- Remaining gap.
- Dependency or blocker.
- Whether continued investment is justified.

Phase 3 — Analyze throughput Measure or defensibly estimate:

- Verified deliverables completed.
- Partially completed work.
- Unverified completion claims.
- Reports or plans produced.
- Repeated or superseded work.
- Failed or inconclusive verification attempts.
- Work-in-progress count.
- Rework ratio.
- Time or effort spent blocked.
- Token cost per verified deliverable, only if actual token evidence exists.

Phase 4 — Perform root-cause analysis Create a ranked causal analysis rather than a flat problem list.

For each root cause include:

- Root cause.
- Symptoms it explains.
- Concrete evidence.
- Affected workstreams.
- Impact.
- Confidence.
- Controllability.
- Corrective action.
- Expected unlock.
- User decision required, if any.

Phase 5 — Define the recovery sequence Produce:

A. Immediate containment

- Work that should stop now.
- Work that should continue.
- Work that should be archived, superseded, or consolidated.
- Decisions required before additional tokens are spent.

B. Next three vertical slices For each slice specify:

- User or operational outcome.
- Exact scope.
- Dependencies.
- Likely files or modules involved.

- Explicit non-goals.
- Acceptance criteria.
- Focused verification commands.
- Evidence artifact required.
- Maximum reasonable discovery/planning allowance.
- Stop or escalation condition.

C. Ten-working-day recovery plan

- Sequence work according to dependencies.
- Permit only one primary implementation slice at a time unless independence is proven.
- Assign measurable daily or task-level checkpoints.
- Define when to stop, continue, escalate, or request a decision.
- Avoid committing to dates unsupported by repository evidence.

D. Sustainable operating model Define a lightweight execution contract for future tasks:

- One bounded outcome per task.
- Named files or modules in scope.
- Explicit acceptance criteria before implementation.
- Evidence reuse before new discovery.
- Focused verification before completion claims.
- No broad refactor without evidence and approval.
- No second audit of the same area without new evidence or a changed question.
- Every task ends as completed, partially completed, genuinely blocked, or deliberately stopped.
- Track tokens against verified deliverables rather than activity volume.

Required final report:

1. Executive verdict Answer directly:

- Are we progressing meaningfully?
- Is token consumption proportionate to verified advancement?
- What is the single biggest reason for the current result?
- What must change immediately?

2. Evidence limitations State what could and could not be proven, especially concerning token consumption and task history.

3. Progress scorecard Score from 0–5, with evidence:

- Product-value delivery.
- Vertical-slice completion.
- Architecture convergence.
- Implementation throughput.
- Verification discipline.
- Rework control.
- Blocker resolution.
- Token efficiency.

- Release readiness.
- Operational readiness.

Do not calculate a false overall score when material evidence is missing.

4. Work-state register Separate:

- Completed.
- Partially completed.
- Blocked.
- Unverified.
- Superseded.
- Not started but still justified.
- Work that should be stopped.

5. Ranked root-cause matrix Rank by:

- Impact.
- Evidence strength.
- Frequency.
- Controllability.
- Expected recovery value.

6. Recovery plan Include immediate containment, the next three vertical slices, the ten-working-day sequence, and the sustainable execution protocol.

7. Decision register List only decisions that genuinely require user input. For each decision provide:

- The question.
- Why it matters now.
- Available options.
- Tradeoffs.
- Recommended option.
- Consequence of delaying it.

8. Immediate actions End with exactly the three highest-impact actions to take next. Each action must have:

- A concrete owner or decision-maker.
- A measurable output.
- A verification method.
- A stop condition.
- A reason it outranks the remaining work.

Expected artifact: Save the complete assessment to:

`what-next/platform-progress-recovery-assessment-YYYY-MM-DD.md`

Also provide a concise executive summary in chat.

Focused verification: Inspect `package.json` before running scripts and use only commands that actually exist. Record each command as passed, failed, skipped, timed out, or blocked.

Potential commands, when applicable:

- `npm run typecheck`
- `npm run policy:gates`
- `npm run workflow:assurance:runtime-check`
- `npm run prisma:validate`
- Focused tests for workstreams claimed as completed
- `npm run build:app` only when build or release readiness is being evaluated

A failed command is evidence, not permission to modify the repository.

Risk controls:

- Preserve tenant isolation, RBAC, module entitlement, segregation of duties, auditability, redaction, and server-owned business truth.
- Do not expose secrets, tokens, personal data, payroll information, provider credentials, or sensitive audit evidence.
- Do not make destructive database, migration, seed, git, or filesystem changes.
- Do not overwrite or revert unrelated dirty-worktree changes.
- Do not confuse planning artifacts with delivered capabilities.
- Do not classify ordinary engineering difficulty as an external blocker.
- Do not recommend a broad redesign without demonstrating why smaller corrections cannot solve the identified cause.
- Do not claim legal, accounting, security, privacy, accessibility, or release certification.
- Do not recommend more agents, prompts, reports, or orchestration layers unless evidence shows they reduce a specific bottleneck.

Success criteria: This assessment is complete only when:

- Every material conclusion cites repository, history, command, report, or task evidence.
- Facts, inferences, hypotheses, and unknowns are visibly distinguished.
- Token-efficiency conclusions disclose whether actual token data was available.
- Existing work is separated into completed, partial, blocked, unverified, superseded, and stopped states.
- The highest-impact root causes are ranked rather than merely listed.
- Every genuine blocker has a precise unblock condition and owner.
- The recovery plan identifies one critical path and three bounded vertical slices.
- Each proposed slice has acceptance criteria, verification, non-goals, and a stop condition.
- The plan explicitly removes or pauses low-value work.
- The final three actions are immediately executable and measurable.
- No application code or unrelated files were changed.

Non-goals:

- Do not implement the proposed recovery plan.
- Do not perform a platform redesign.
- Do not refactor unrelated code.
- Do not fix unrelated lint, typecheck, test, or build failures.
- Do not create speculative modules or features.

- Do not generate another generic roadmap.
- Do not hide uncertainty behind confident language.
- Do not optimize for visible activity; optimize for verified operational progress.